SunCast: Forecasting Rooftop Solar Output and Savings in Pakistan

Deliverables 1 and 2: Data Pipeline, Models and Evaluation

Anoosha Rizwan Vohra (24L-3096) Kainat Afzal (24L-3084)
Manumha Nadeem (24L-3097)

November 10, 2026

Contents

1 Introduction **3**

2 Problem Statement **3**

3 Target Audience **4**

4 AI Justification **4**

5 Data Collection **4**
5.1 Study area and period . . . 5
5.2 NASA POWER (sunlight, temperature, humidity, wind) . . . 5
5.3 Open-Meteo (cloud cover and smog) . . . 5
5.4 Merging weather sources . . . 6
5.5 Kaggle solar plant data . . . 6
5.6 Retailer list . . . 6
5.7 Reproducibility . . . 6

6 Data Cleaning **7**
6.1 NASA POWER weather data . . . 7
6.2 Kaggle plant data . . . 7

7 Feature Strategy **8**

8 Exploratory Data Analysis **8**
8.1 Sunlight by month and city . . . 8
8.2 Correlation between weather features . . . 9
8.3 Sunlight and panel output . . . 10
8.4 Cloud cover and sunlight . . . 11
8.5 Missing values . . . 11
8.6 What the analysis tells us . . . 12

9 Model Architecture and Implementation **12**
9.1 Frameworks and Task . . . 12
9.2 Inputs and Target . . . 12
9.3 Neural Network . . . 12
9.4 Hyperparameters of the Neural Network . . . 13
9.5 Gradient Boosting (Comparison Model) . . . 14

9.6 Hyperparameters of Gradient Boosting 14
9.7 Model Comparison on the Validation Set 14

10 Validation Strategy **15**

11 Physics Baseline **15**

12 Evaluation **17**
12.1 Test set and metrics 17
12.2 Test results 17

13 Error Analysis **18**
13.1 Error by hour of the day 18
13.2 Error by sunlight level 19
13.3 Largest errors 19

14 Bias Check **20**
14.1 By inverter 20
14.2 By month 21
14.3 Summary of the bias check 21

15 Limitations **21**

16 Contribution Table **22**

17 Next Steps **22**

1 Introduction

Rooftop solar has become one of the most talked about ways for households in Pakistan to deal with rising electricity bills. Installing a system is a large investment, but most people decide with little more than a salesperson's estimate or an online calculator that uses yearly averages. Those estimates ignore the things that actually change how much power panels produce from day to day, such as clouds, smog, heat, and the climate of each city.

SunCast is a web application that aims to close this gap. A user enters their city, the size of their solar system, and their electricity tariff, and the app forecasts how much power the system will generate over the next few days using live weather data and a machine learning model. The forecast is converted into estimated savings in rupees, and a chatbot answers common solar questions and suggests retailers in the user's city.

This report covers the first two deliverables of the project. The first part defines the problem and builds the data pipeline: the problem and target audience, why machine learning is needed, how we collected, cleaned, and prepared the data, and our exploratory analysis. The second part covers the models: their architecture and hyperparameters, how we split the data for training, validation, and testing, a physics baseline to compare against, and the evaluation, error analysis, and bias check. The report ends with the next steps toward the web application.

2 Problem Statement

Electricity prices in Pakistan have gone up sharply over the last few years, and for many households the monthly bill has become one of their biggest expenses. Because of this, more and more people are looking at rooftop solar as a way to cut costs. The problem is that installing solar is a big financial decision, and most people make it with very little reliable information about what the system will actually do for them.

Most solar calculators available online work with yearly averages. They take the size of the system, multiply it by an average amount of sunlight for the region, and give a single number. This approach misses several things that have a real effect on how much power panels produce in Pakistan:

- **Clouds and weather:** output changes a lot from day to day, and a yearly average says nothing about what to expect this week.
- **Smog and dust:** in cities like Lahore, winter smog blocks a noticeable share of sunlight for weeks at a time, which simple calculators ignore completely.
- **Heat:** solar panels become less efficient as they get hotter, so the hottest months do not produce as much as the strong sunlight would suggest.
- **Location:** sunlight and weather patterns differ between cities like Karachi, Lahore, and Peshawar, but many tools treat the whole country the same way.

This creates two problems. People who are thinking about buying solar cannot tell how much they will really save or whether the investment is worth it. People who already have solar cannot tell whether their system is performing as it should, or plan their electricity use around the days and hours when generation will be high.

On top of this, the practical information people need is scattered. Common questions about inverter types, whether a battery is worth the cost, how net metering works, and which retailers in their city are trustworthy are usually answered through word of mouth or sales pitches, which may not be accurate or unbiased.

SunCast addresses these problems with a web application that forecasts solar generation for a specific location using live weather data and a machine learning model trained on historical

sunlight, weather, and air quality data. The forecast is converted into estimated savings in rupees, and a built in chatbot answers common solar questions and suggests retailers in the user's city.

3 Target Audience

SunCast is designed for the following groups of users:

- **Homeowners considering solar:** people who want to know how much a system of a given size would generate at their location and how much money it would save before they spend on an installation.
- **Existing solar owners:** people who already have panels and want to know what to expect over the next few days, check whether their system is performing normally, and plan heavy electricity use (such as running air conditioners, washing machines, or water pumps) for high generation hours.
- **Small businesses:** shops, clinics, schools, and offices that run during daytime hours and want to understand how much of their electricity demand solar could cover.
- **First time buyers with little technical knowledge:** users who do not understand terms like inverter capacity or net metering and need simple, trustworthy answers before talking to a retailer.

For the first version, the app covers five major cities: Lahore, Karachi, Islamabad, Peshawar, and Multan. These cities were chosen because they cover different climates across the country, from the humid coastal weather of Karachi to the smoggy winters of Lahore and the hotter, drier conditions of Multan. The system is built so that more cities can be added later by including their coordinates in the data pipeline.

4 AI Justification

Solar output depends on many factors that affect each other in complicated, nonlinear ways: cloud cover, smog and dust that block sunlight, panels losing efficiency when they get hot, and changes between seasons. Our analysis supports this: sunlight varies strongly by month and city, and cloud cover and PM2.5 each reduce it. A fixed formula or rule-based program uses fixed averages, so it cannot capture these effects together. A machine learning model can learn them directly from real data and give a more accurate forecast for each city.

The chatbot also needs AI. People ask solar questions (inverters, batteries, net metering) in plain, everyday language, and rule-based code cannot handle so many different ways of asking the same question.

5 Data Collection

SunCast needs three kinds of data: weather and sunlight for Pakistani cities, smog, and real solar panel output. Four sources cover them (Table 1). All collection code is in the `data_scripts/` folder of our repository, and the scripts write log files that we kept as evidence of each download.

Source	What it gives us	How we got it	Script
NASA POWER hourly API	Sunlight (W/m^2), temperature, humidity, wind speed	API, no key needed	`fetch_nasa_power.py`
Open-Meteo Historical Weather API	Cloud cover (%)	API, no key needed	`fetch_openmeteo.py`
Open-Meteo Air Quality API	PM2.5 ($\mu g/m^3$), our measure of smog	API, no key needed	`fetch_openmeteo.py`
Kaggle "Solar Power Generation Data"	Real power output of a solar plant, with its own weather sensor readings	Download from Kaggle	`load_kaggle_generation.py`
Retailer list	Solar retailers by city	Compiled by hand by the team	`data/retailers.csv`

Table 1: Data sources used in SunCast.

5.1 Study area and period

We collected data for five cities that together cover very different climates: Lahore, Karachi, Islamabad, Peshawar and Multan. Each city is represented by one coordinate point, and all APIs were queried for the same points. We used the full year 2024, hourly, which gives $5 \times 8{,}784 = 43{,}920$ hourly records (2024 was a leap year).

5.2 NASA POWER (sunlight, temperature, humidity, wind)

`fetch_nasa_power.py` requests four variables from the NASA POWER hourly point API with the renewable energy community setting: all-sky surface sunlight (`ALLSKY_SFC_SW_DWN`, which already accounts for clouds), air temperature at 2 m, relative humidity at 2 m and wind speed at 10 m. The script asks for one month at a time for each city, retries a request up to three times if it fails, and pauses one second between requests to avoid overloading the service. NASA returns times in UTC, so the script converts them to Pakistan time (Asia/Karachi) before saving. Missing readings are marked with the code -999 in the API output, and our cleaning step replaces them (see the Data Cleaning section). The log (`data_scripts/logs/fetch_nasa_power.log`) records every request: all 12 months were downloaded for all 5 cities, giving 8,784 rows per city.

5.3 Open-Meteo (cloud cover and smog)

NASA POWER does not give cloud cover or air quality, so `fetch_openmeteo.py` adds them from two Open-Meteo APIs: the Historical Weather API for hourly cloud cover, and the Air Quality API for hourly PM2.5. Both are requested in the Asia/Karachi time zone so their timestamps line up with the NASA data. Because the NASA times were shifted from UTC to Pakistan time (five hours ahead), the last hours of December 2024 fall on 1 January 2025. The script therefore asks for one extra day, which is why each city has 8,808 Open-Meteo rows (366 days plus one extra day). Table 2 summarises the evidence from the log files.

Step	Rows	Log file
NASA POWER, per city	8,784	`fetch_nasa_power.log`
NASA POWER, five cities, cleaned	43,920	`clean_nasa_report.txt`
Open-Meteo, per city	8,808	`fetch_openmeteo.log`
After merge with NASA data	43,920	`fetch_openmeteo.log`
Missing cloud cover / PM2.5 after merge	0 / 0	`fetch_openmeteo.log`

Table 2: Row counts recorded by the collection scripts.

5.4 Merging weather sources

The NASA data and the Open-Meteo data are joined on city and timestamp. We used a left join so that every NASA row is kept, and added cloud cover and PM2.5 where the timestamps match. All 43,920 rows matched, with no missing cloud cover or PM2.5 values. The script also checks that cloud cover stays between 0 and 100% and that PM2.5 is not negative (any invalid value would be set to missing and filled only if the gap is three hours or shorter), but no value needed fixing. The script saves two versions of the merged table:

- `merged_weather.csv`, which keeps the city name, used for exploratory analysis;
- `merged_weather_encoded.csv`, where the city column is one hot encoded into five 0/1 columns (`city_Lahore`, `city_Karachi`, and so on), used for modelling.

5.5 Kaggle solar plant data

Weather data alone cannot tell us how much power a panel produces, so we need a measured output as the training target. We use the public Kaggle dataset *Solar Power Generation Data* by Ani Kannal (https://www.kaggle.com/datasets/anikannal/solar-power-generation-data). It contains two files for one solar plant: `Plant_1_Generation_Data.csv` (power output of each inverter, recorded every 15 minutes) and `Plant_1_Weather_Sensor_Data.csv` (sunlight, ambient and module temperature measured at the plant). The Kaggle data comes from a plant in India in 2020 and the file has no city column, so it is used to learn how weather turns into power, while the city-specific weather features come from NASA POWER and Open-Meteo. `load_kaggle_generation.py` loads both files and prints their shapes.

5.6 Retailer list

For the app's retailer suggestions we compiled a directory of 39 solar retailers and installers across the five study cities (8 in Lahore, 8 in Karachi, 8 in Islamabad, 7 in Peshawar and 8 in Multan). Each entry has four columns: name, area, contact and rating. Business names, areas, phone numbers and customer ratings were collected by hand from public Google Maps listings in October 2026. The directory is stored in `data/retailers.csv` and will be used by the web app to suggest local retailers to users.

5.7 Reproducibility

Large data files are not stored in the repository (they are excluded by `.gitignore`). Anyone can recreate them by running the scripts in this order: `fetch_nasa_power.py`, `clean_nasa.py`, `fetch_openmeteo.py`. The repository is at https://github.com/anushavohra/suncast, and a link to the cleaned NASA data is in the README.

6 Data Cleaning

We cleaned two datasets: the NASA POWER weather data (`clean_nasa.py`) and the Kaggle solar plant data (`clean_generation.py`). The Open-Meteo values were validated during the merge, as described in the Data Collection section. Each script prints a before and after summary, and the NASA script saves it to `data_scripts/logs/clean_nasa_report.txt`.

6.1 NASA POWER weather data

`clean_nasa.py` combines the hourly files for all five cities into one table, renames the NASA variable codes to readable names (irradiance, temperature, humidity, and wind speed), and applies the following steps:

- **Missing values:** NASA POWER marks missing readings with the code −999. These are replaced with empty values, and gaps of up to three hours are filled by time based interpolation, one city at a time so cities never mix. Longer gaps are dropped.
- **Duplicates:** rows with the same city and timestamp are removed.
- **Impossible values:** negative irradiance, and any irradiance between 21:00 and 04:00 when the sun is always down, are set to zero. Irradiance above the solar constant (about 1361 W/m^2), temperature outside −20 to 55 °C, humidity outside 0 to 100%, and wind speed outside 0 to 40 m/s are treated as errors.
- **Statistical outliers:** values more than 1.5 times the interquartile range beyond the quartiles are counted but kept, because heatwaves and windy days are real weather that the model needs to learn from.

Table 3 shows the results. The 2024 NASA POWER data passed every check, with no missing values, duplicates, or physically impossible readings, which confirms it as a reliable source. The only notable finding was 1,931 statistical outliers in wind speed. Wind speed is usually low with occasional strong gusts, so the interquartile range rule flags many real windy and stormy hours; we kept them on purpose.

Check	Count
Rows before cleaning	43,920
Missing values (all four columns)	0
Duplicate rows removed	0
Impossible values fixed	0
Wind speed outliers (kept)	1,931
Temperature and humidity outliers	0
Rows after cleaning	43,920

Table 3: Results of cleaning the NASA POWER data (five cities, 2024).

6.2 Kaggle plant data

`clean_generation.py` cleans the Kaggle plant data and merges the generation file with the weather sensor file on the timestamp.

- Missing values: none found.
- Duplicate rows (same timestamp and inverter): none found.
- Negative power values: none found.

- Merge with the sensor data: 68,778 rows became 68,774 rows, because 4 generation rows had no matching sensor reading.
- Inverter faults: 63 rows where sunlight was present (irradiation above 0.2) but the inverter output was zero were removed as outliers.
- Night readings: no rows had sunlight above zero at night.

The cleaned Kaggle table has 68,711 rows.

7 Feature Strategy

`feature_engineering.py` prepares the cleaned Kaggle table for modelling.

- **Sine/cosine encoding** of hour and month, because time is a cycle (hour 23 is close to hour 0, and December is close to January).
- **One hot encoding** of the inverter ID (22 inverters). The city is one hot encoded separately in the merged weather table (see the Data Collection section), because the Kaggle data has no city column.
- **StandardScaler** on ambient temperature, module temperature and irradiation, so that all inputs have mean 0 and standard deviation 1. DC and AC power are the targets, so they are not scaled.
- `DAILY_YIELD` and `TOTAL_YIELD` were removed because they already contain the power result and would let the model cheat.

The final feature table has 68,711 rows and 31 columns.

8 Exploratory Data Analysis

Before building any model we looked at the data to check that it behaves the way physics says it should, and to find which inputs matter. All charts are produced by `data_scripts/eda.py` from the merged weather table (43,920 hourly rows, five cities, 2024) and from the cleaned Kaggle plant data.

8.1 Sunlight by month and city

Figure 1 shows the average daily sunlight energy for each month. All five cities follow the same shape: sunlight rises from winter to a peak in May or June, then falls again. The peak is between 7.2 and 7.5 kWh/m^2/day for every city (May for Islamabad, Karachi, Lahore and Multan, June for Peshawar).

The cities differ most in winter and during the monsoon. Lahore and Multan fall to about 1.7 and 1.9 kWh/m^2/day in January, roughly a quarter of their May values. This is consistent with winter fog and smog in the Punjab, although this chart alone does not prove the cause. Karachi stays much more even through the year; its lowest month is August (4.1 kWh/m^2/day), which fits the cloudy monsoon season. Over the full year, Karachi receives the most sunlight (5.31 kWh/m^2/day), followed by Peshawar (5.06), Islamabad (4.80), Multan (4.77) and Lahore (4.72). Two points matter for the product: location and season change the expected output a lot, so a single fixed average cannot be right for every user, and the winter dip in Lahore and Multan is exactly the kind of effect a data-driven model should capture.

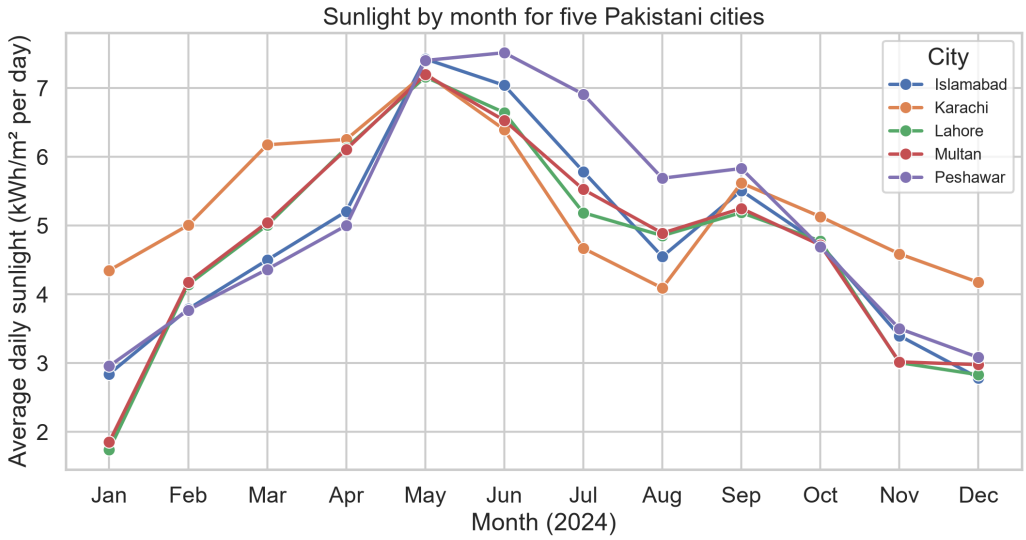


Figure 1: Average daily sunlight energy by month for the five cities (2024, NASA POWER).

8.2 Correlation between weather features

Figure 2 shows how the weather variables relate to each other during daytime hours (night hours are excluded, because sunlight is always zero then and would distort the picture). Sunlight is positively related to temperature ($r = 0.45$) and negatively related to humidity ($r = -0.42$): sunny hours are warmer and drier. Cloud cover and PM2.5 both reduce sunlight, but only mildly in this hour-by-hour view ($r = -0.20$ and $r = -0.18$). These links look weak because each hourly value also depends on the time of day and the height of the sun, which the correlation does not remove. They are also not strong enough to say that cloud cover or smog can be ignored; Figure 4 isolates their effect. No pair of input variables is so strongly correlated that it would make a model unstable (the largest is $|r| = 0.45$).

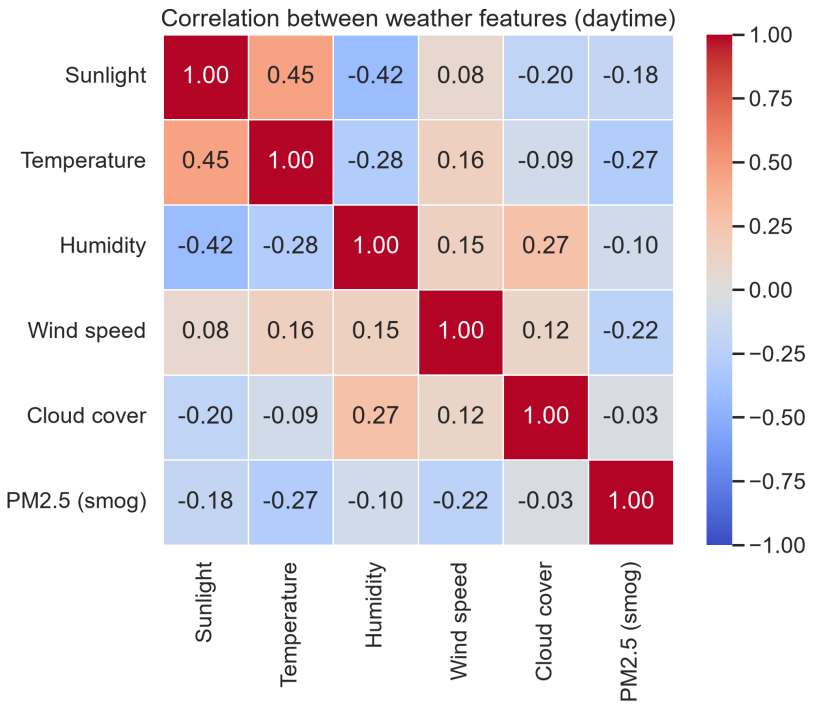


Figure 2: Correlation between the weather features for daytime hours.

8.3 Sunlight and panel output

Figure 3 compares sunlight with the DC power produced by the inverters in the Kaggle plant data. The two are almost perfectly linked, with a correlation of r = 0.993 across 68,711 cleaned readings. This confirms that sunlight is the main driver of solar output, so accurate sunlight forecasts will translate directly into accurate generation forecasts.

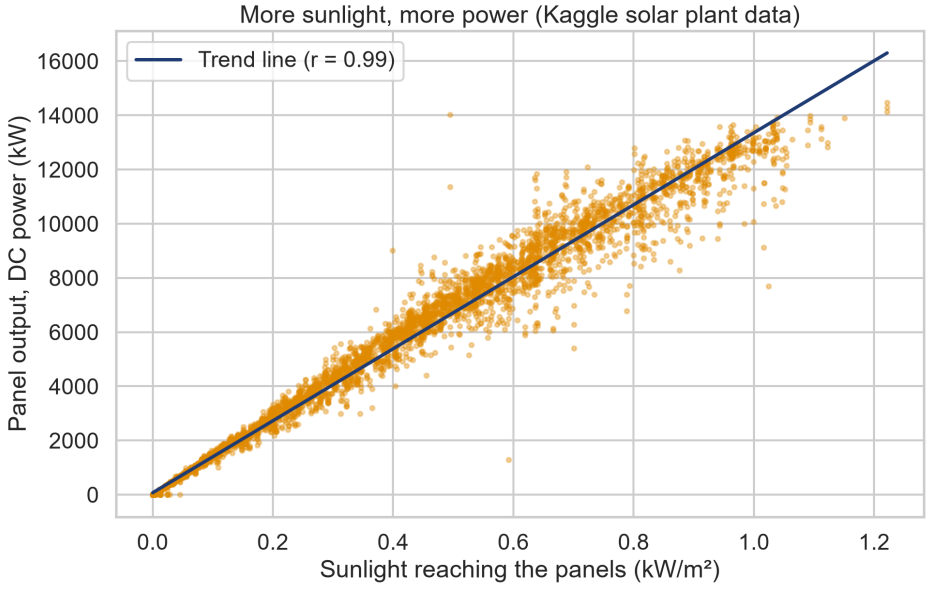


Figure 3: Sunlight against panel output (DC power) in the Kaggle solar plant data.

8.4 Cloud cover and sunlight

The Kaggle plant data does not record cloud cover and comes from a different place and year, so we cannot plot cloud cover against panel output directly. Instead we show the effect in two steps: Figure 4 shows how cloud cover changes sunlight, and Figure 3 shows how sunlight changes output. For Figure 4 we only use the hours from 11:00 to 13:00, when the sun is highest, so that the effect of the time of day is mostly removed.

Midday sunlight stays fairly steady while cloud cover rises up to about 80%, and drops clearly under heavy overcast. At 90 to 100% cloud cover the average is about 490 W/m^2, against about 730 W/m^2 under clear skies (below 10% cloud), a fall of 32%. The correlation between cloud cover and midday sunlight is moderate ($r = -0.41$). The wide spread inside each band has two main causes: the sun sits lower in winter than in summer even at midday, and cloud cover and sunlight come from two different models (Open-Meteo and NASA), which do not always agree. For the project this means cloud cover is a useful but imperfect input, and it supports using several weather inputs together instead of a single formula.

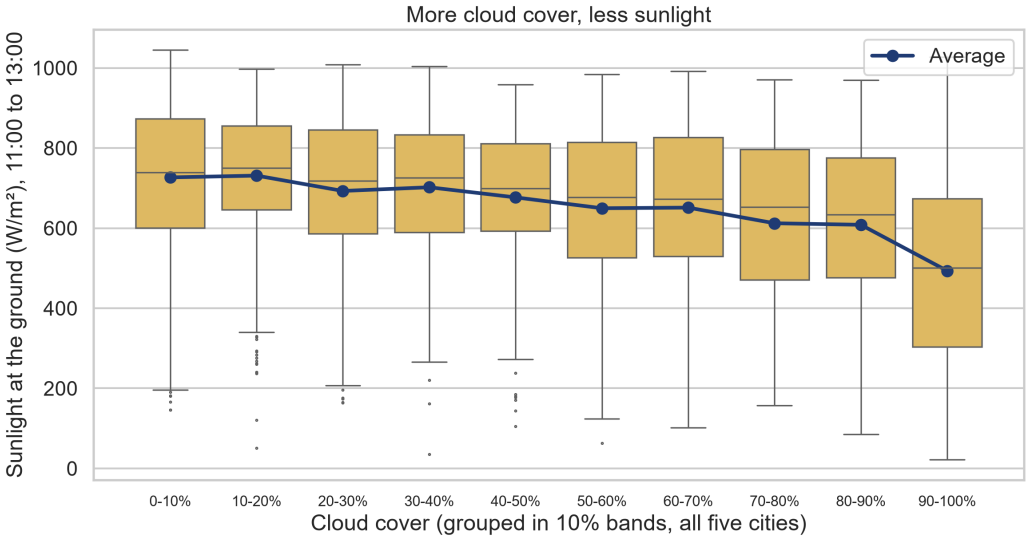


Figure 4: Midday sunlight (11:00 to 13:00) for each 10% band of cloud cover, all five cities. The line connects the band averages.

8.5 Missing values

Figure 5 shows the share of missing values in each raw weather column before cleaning. We found none in the NASA POWER columns (after treating the −999 code as missing) or in the Open-Meteo columns, so none of these values had to be filled in. The cleaning of the Kaggle plant data (faulty inverters and invalid night readings) is described in the Data Cleaning section.

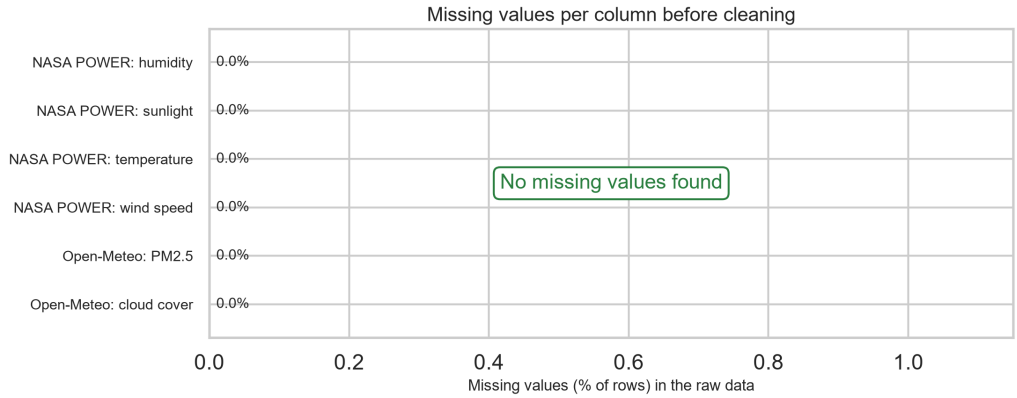


Figure 5: Missing values per column in the raw data.

8.6 What the analysis tells us

Sunlight is the main driver of output and varies strongly by month and city. Temperature, humidity, cloud cover and smog each have a smaller effect, and they act together, which supports using a machine learning model with all of them as inputs. The city must also be an input, and this is why we one hot encode it.

9 Model Architecture and Implementation

9.1 Frameworks and Task

The task is regression: predict the DC power (kW) of a solar plant from weather and time inputs. We built two models and compared them: a neural network in PyTorch (`src/nn_model.py`) and a gradient boosting model in Scikit-Learn (`src/model.py`). The hyperparameter searches are in `src/tune.py` and `src/nn_model.py`.

9.2 Inputs and Target

There are 29 input features: ambient temperature, module temperature and irradiation (scaled); sine and cosine of hour and month (4); and the one hot encoded inverter ID (22). The target is `DC_POWER`. `AC_POWER` is not used as an input because it is almost the same as the target and would let the model cheat. Both models use exactly the same train/validation/test split, which is described in the next section. The validation set was used for tuning; the test set was kept for the final evaluation.

9.3 Neural Network

The network is a fully connected feed-forward network with two hidden layers (Figure 6).

- **Layers:** 29 inputs, then two hidden layers of 64 units each with ReLU activation and dropout, then 1 output unit.
- **Loss function:** mean squared error (MSE). The target was divided by 1000 so that the loss values stay small.
- **Optimizer:** Adam.
- **Training:** 30 epochs, mini-batches, random seed 42.

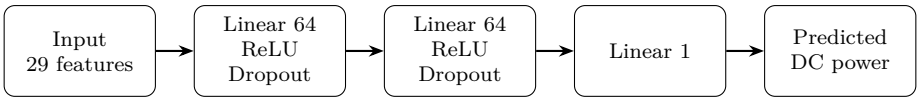


Figure 6: Architecture of the neural network.

9.4 Hyperparameters of the Neural Network

We tested two learning rates, two batch sizes and two dropout values (8 settings). All results are on the validation set (Table 4).

Table 4: Neural network hyperparameters tested (best row in bold).

Learning rate	Batch size	Dropout	MAE	RMSE	R^2
0.001	**64**	**0.0**	**174.37**	**398.71**	**0.9902**
0.001	64	0.2	202.10	404.78	0.9899
0.001	256	0.0	178.59	399.05	0.9902
0.001	256	0.2	258.08	429.08	0.9887
0.01	64	0.0	177.15	399.98	0.9902
0.01	64	0.2	251.46	434.28	0.9884
0.01	256	0.0	197.43	401.48	0.9901
0.01	256	0.2	419.19	528.50	0.9829

The best setting (learning rate 0.001, batch size 64, dropout 0.0) had the lowest validation RMSE (398.71). Dropout of 0.2 made the results worse in every pair. Dropout is meant to reduce overfitting, but our data is large (48,097 training rows) and has few input features, so the network had little to overfit and the extra noise only removed useful information. Figure 7 shows the training and validation loss of the best setting.

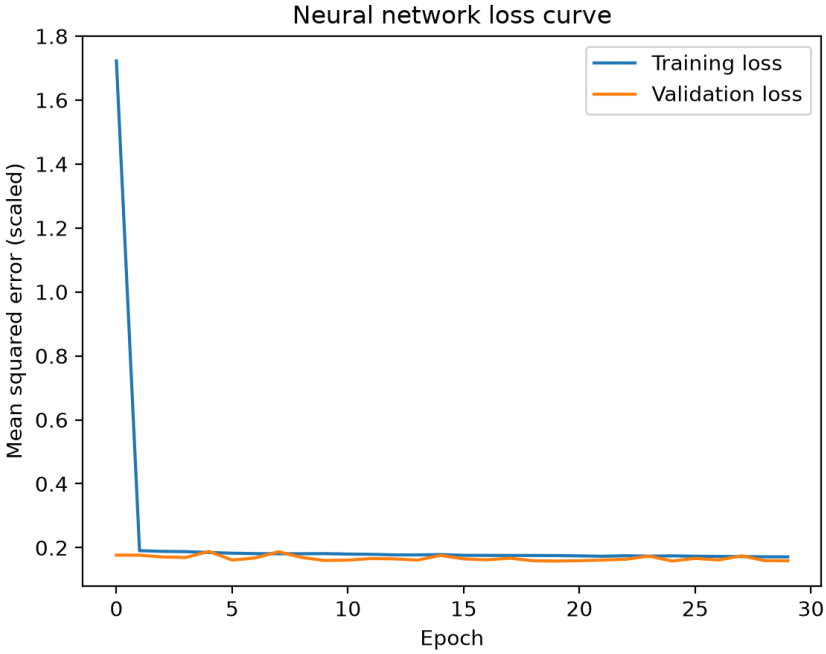


Figure 7: Training and validation loss of the best neural network.

The training loss starts high in the first epoch because it is averaged over the early, untrained updates, and then drops quickly. After about two epochs the training and validation losses are

almost the same and stay flat, so the network learned the pattern quickly and does not overfit: the validation loss does not rise while the training loss falls.

9.5 Gradient Boosting (Comparison Model)

As a second model we used `HistGradientBoostingRegressor`. It builds many small decision trees one after another, and each new tree learns to correct the mistakes of the trees before it. It uses squared error loss and adds each tree in the direction that reduces the loss most (gradient boosting). We used it to check whether a simpler tree-based model can match the neural network on this tabular data.

9.6 Hyperparameters of Gradient Boosting

We tested the learning rate, the maximum tree depth and the number of boosting iterations (18 settings, Table 5).

Table 5: Gradient boosting hyperparameters tested (best row in bold).

Learning rate	Max depth	Iterations	MAE	RMSE	R^2
0.05	3	100	188.36	405.06	0.9899
0.05	3	300	169.06	390.30	0.9906
0.05	6	100	172.63	384.05	0.9909
0.05	6	300	154.47	368.91	0.9916
0.05	None	100	171.67	377.83	0.9912
0.05	None	300	156.19	370.26	0.9916
0.1	3	100	172.40	393.93	0.9905
0.1	3	300	166.25	383.65	0.9910
0.1	6	100	155.87	371.17	0.9915
0.1	6	300	153.35	366.19	0.9918
0.1	None	100	155.57	369.90	0.9916
0.1	None	300	155.29	369.68	0.9916
0.2	3	100	170.34	385.84	0.9909
0.2	3	300	162.02	373.65	0.9914
0.2	6	100	152.48	365.27	0.9918
0.2	**6**	**300**	**151.82**	**363.48**	**0.9919**
0.2	None	100	155.75	370.53	0.9916
0.2	None	300	155.75	370.53	0.9916

The best setting (learning rate 0.2, maximum depth 6, 300 iterations) had the lowest validation RMSE (363.48) and the highest R^2 (0.9919). Shallow trees (depth 3) were clearly worse, so the model needs some depth to capture how sunlight and temperature work together. Unlimited depth did not help, so a depth of 6 is enough. The differences between the best settings are small, so the model is not very sensitive to these choices.

9.7 Model Comparison on the Validation Set

Table 6: Best setting of each model on the validation set.

Model	MAE	RMSE	R^2
Neural network (lr 0.001, batch 64, dropout 0)	174.37	398.71	0.9902
Gradient boosting (lr 0.2, depth 6, 300 iterations)	151.82	363.48	0.9919

Both models fit the data very well. Gradient boosting was slightly better on every metric, which is common for tabular data with few features. The final comparison on the unseen test set is in the evaluation section.

10 Validation Strategy

We split the cleaned and feature engineered Kaggle table (68,711 rows) into three parts: 70% for training, 15% for validation, and 15% for testing. The split is done in two steps with `train_test_split` from Scikit-Learn and a fixed seed (`random_state=42`): first 30% of the rows are held out, and then that 30% is cut in half into the validation and test sets. Using a fixed seed means every run, and every member's code, gets exactly the same rows in each set.

Each set has a clear job:

- **Training set:** used to fit the models (and the single constant of the physics baseline).
- **Validation set:** used to compare hyperparameter settings and pick the final ones.
- **Test set:** kept aside until the very end and used once to report the final results, so the reported numbers are not influenced by tuning.

The script `src/split_data.py` creates the split, checks that no row appears in more than one set, and confirms that it is identical to the split used by the models in `src/model.py`. This guarantees that the gradient boosting model, the neural network, and the physics baseline are all tested on exactly the same rows.

Table 7 shows that the three sets look alike. The average DC power differs by less than 0.4% between them, and the share of zero output rows (mostly night time and the edges of the day) is about 46% in each. This means the test set is a fair sample of the data and none of the sets is easier or harder by chance.

Set	Rows	Share	Mean DC power	Std. dev.	Zero output
Train	48,097	70.0%	3149.91	4039.76	46.4%
Validation	10,307	15.0%	3145.62	4035.85	46.6%
Test	10,307	15.0%	3155.23	4026.74	46.2%

Table 7: Size and target statistics of the three sets (from `results/split_summary.csv`).

We chose a single fixed split instead of K fold cross validation for two reasons. The dataset is large enough (over 10,000 rows in each of the validation and test sets) for one split to give stable results, and training the neural network many times for every hyperparameter setting would have taken much longer. The main weakness of a random split on 15 minute data is that neighbouring readings, which are very similar, can end up in different sets; this is discussed in the Limitations section.

11 Physics Baseline

To show whether machine learning actually adds value, we compare our models against the standard way solar output is estimated without ML. The baseline in `src/physics_baseline.py` uses a fixed formula:

$$P = k \cdot G \cdot (1 + \gamma\,(T_{module} - 25)) \tag{1}$$

where P is the predicted DC power, G is the irradiation, T_{module} is the panel temperature in °C, and $\gamma = -0.004$, a typical value for silicon panels. In words, output grows in proportion to

sunlight and drops by about 0.4% for every degree the panel is hotter than 25 °C. The constant k represents the size of the plant. It is the only number the baseline learns, and it is fitted once on the training set by least squares. The baseline uses the original, unscaled sunlight and temperature values, and is evaluated on the same validation and test rows as the ML models with the same MAE, RMSE, and R^2 functions.

We also tested a simpler version that uses sunlight only, without the temperature correction, to see how much the temperature term helps. Table 8 shows the test results next to the two ML models.

Model (test set)	**MAE**	**RMSE**	**R^2**
Physics baseline (sunlight only)	234.89	503.25	0.9844
Physics baseline (sunlight + temperature)	208.32	466.90	0.9866
Neural network	178.26	417.49	0.9892
Gradient boosting	153.59	369.18	0.9916

Table 8: Physics baselines compared with the ML models on the test set.

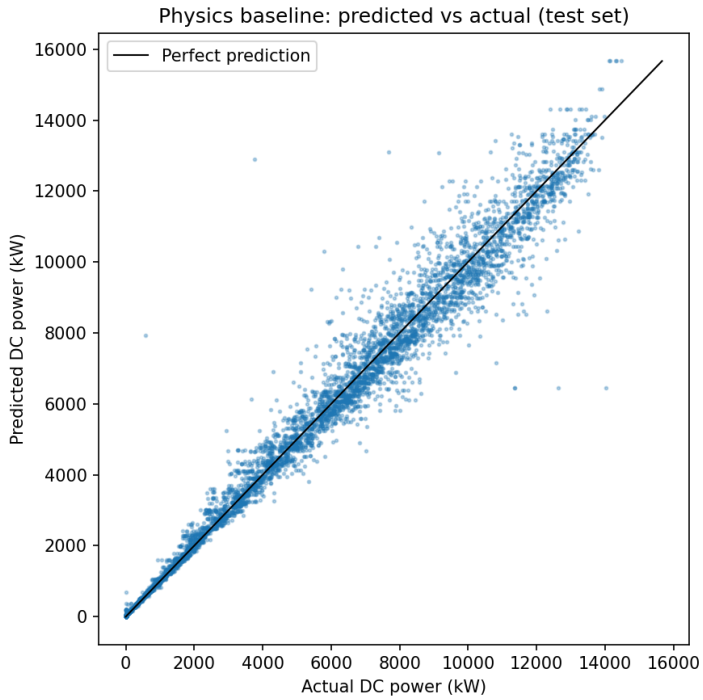


Figure 8: Predicted against actual DC power for the physics baseline (sunlight + temperature) on the test set.

Three things stand out from these results:

- **The baseline is already strong.** An R^2 of 0.987 matches what our exploratory analysis found: sunlight alone explains almost all of the change in output ($r = 0.993$). Any useful model has to beat this, which makes it an honest comparison.
- **The temperature correction helps.** Adding it lowers the MAE from 234.89 to 208.32 (about 11%) and the RMSE by about 7%, which confirms that hot panels really do produce less.
- **Machine learning still does clearly better.** Gradient boosting has about 26% lower MAE and 21% lower RMSE than the best baseline, and the neural network about 14%

lower MAE and 11% lower RMSE. The ML models learn effects the formula cannot, such as differences between inverters and how output changes through the day.

For SunCast, this gap matters because errors in the power forecast carry straight into the savings estimate a user sees. The baseline will also be useful in the app as a fallback and as a sanity check on the model's predictions.

12 Evaluation

12.1 Test set and metrics

The data was split 70/15/15 into training, validation and test sets with a fixed random seed. All hyperparameters were chosen on the validation set. The test set (10,307 rows) was used only once, to produce the numbers in this section, so it shows how the final models perform on data they have not seen.

Our main task is regression: predicting the DC power of the plant. We report three regression metrics:

- **MAE** (mean absolute error): the average size of the error, in the same unit as the power. It is easy to read and explain.
- **RMSE** (root mean squared error): like MAE, but it punishes large mistakes more. This matters for SunCast, because a few large forecast errors hurt the savings estimate more than many small ones.
- R^2: the share of the variation in power that the model explains (1 is perfect).

The course guide also asks for Precision, Recall, F1 and AUC-ROC, which are classification metrics. We therefore added one binary task: is the output *high* or *low*? A row is labelled high when its DC power is above the median DC power of the training set (440.5 kW). The median comes from the training set only, so no test information is used. Because the threshold is a median, the two classes are about the same size, so accuracy alone would not be misleading here. We still report Precision, Recall, F1 and AUC-ROC because they show separately how many low-output hours are wrongly called high (Precision), how many high-output hours are missed (Recall), and how well the predicted power ranks high rows above low rows (AUC-ROC). For the app this matters: calling a low-output hour high makes the savings look better than they are, while missing a high-output hour hides real savings.

12.2 Test results

Table 9 shows the regression results and Table 10 shows the results for the high versus low task.

Model	MAE	RMSE	R^2
Physics baseline (sunlight only)	234.89	503.25	0.9844
Physics baseline (sunlight + temperature)	208.32	466.90	0.9866
Neural network	178.26	417.49	0.9892
Gradient boosting	153.59	369.18	0.9916

Table 9: Regression metrics on the test set (10,307 rows), including the two physics baselines.

Model	Precision	Recall	F1	AUC-ROC
Gradient boosting	0.9973	0.9975	0.9974	1.0000
Neural network	0.9981	0.9959	0.9970	1.0000

Table 10: Metrics for the high versus low output task on the test set (threshold 440.5 kW).

Gradient boosting is the better model on every regression metric. Its MAE is about 14% lower and its RMSE about 12% lower than those of the neural network, and its R^2 is slightly higher. Its test MAE (153.59) is close to its validation MAE (151.82), and the neural network behaves the same way (178.26 on test, 174.37 on validation), so we see no sign that the settings were over-fitted to the validation set. The mean DC power of the test rows is about 3,155, so the average error is about 4.9% of it for gradient boosting and about 5.6% for the neural network. On this evidence we treat gradient boosting as our main model and the neural network as the comparison.

Both models separate high from low output almost perfectly (F1 about 0.997, AUC-ROC 1.00). The differences between them on these metrics are tiny (0.0004 in F1) and should not be read as one model being better. This task is easy: the output is exactly zero at night, and during the day most rows are far above the threshold, so only the rows near sunrise and sunset are close to it. An AUC-ROC of 1.00 therefore does not mean that the models are perfect. The regression metrics and the error analysis in the next section say much more about their quality.

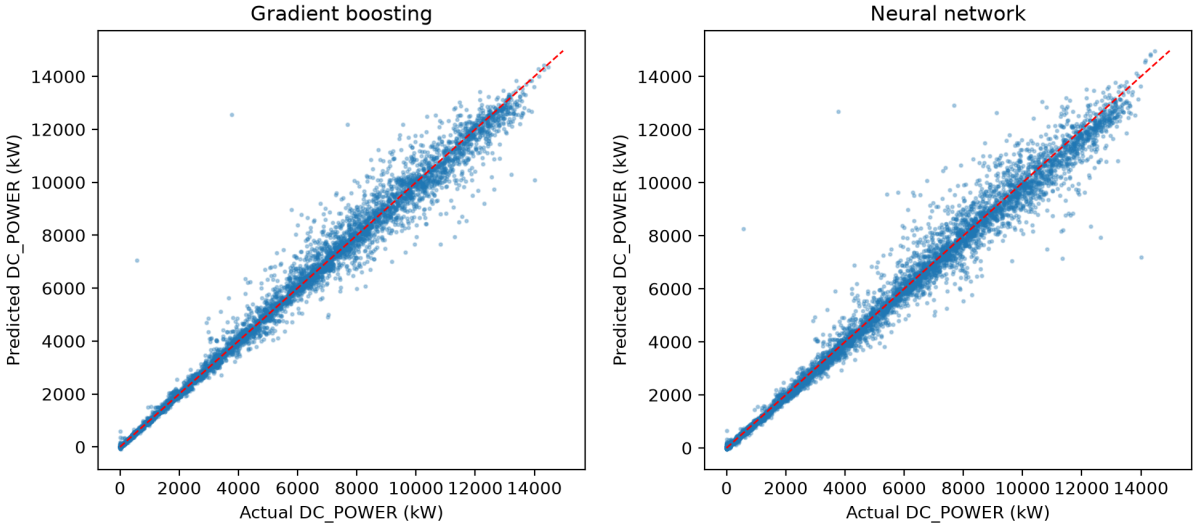


Figure 9: Predicted against actual DC power on the test set for both models. Points on the diagonal line are perfect predictions.

13 Error Analysis

This section looks at where the models make their largest mistakes on the test set. All numbers come from `src/bias_check.py`, and the tables are saved in the `results/` folder. Error means predicted minus actual power, so a negative bias means that the model predicts too little.

13.1 Error by hour of the day

Figure 10 shows the error for each hour. At night (hours 19 to 5) the output is zero and both models predict almost zero, with an average error of only about 2 to 5. In the daytime the absolute error is largest around midday (hours 11 and 12: MAE about 502 for gradient boosting

and 562 to 607 for the neural network), because the output itself is largest then (a mean of about 9,700). As a share of the output, the error at midday is about 5%. The largest relative errors are at the edges of the day: in the hour starting at 18:00 the error is 10.6% of the mean output for gradient boosting and 19.1% for the neural network, and in the hour starting at 06:00 it is 6.0% and 11.2%. In absolute terms these errors are small (24 and 43 at 18:00), so they hardly change the daily energy total, which is what the savings estimate uses.

The signed error shows a difference between the models. Gradient boosting has almost no bias at any hour (the largest is +44 at hour 9, which is 0.6% of the mean output of that hour). The neural network predicts too little in every hour from 6 to 18 except noon, by between 1 and 100.

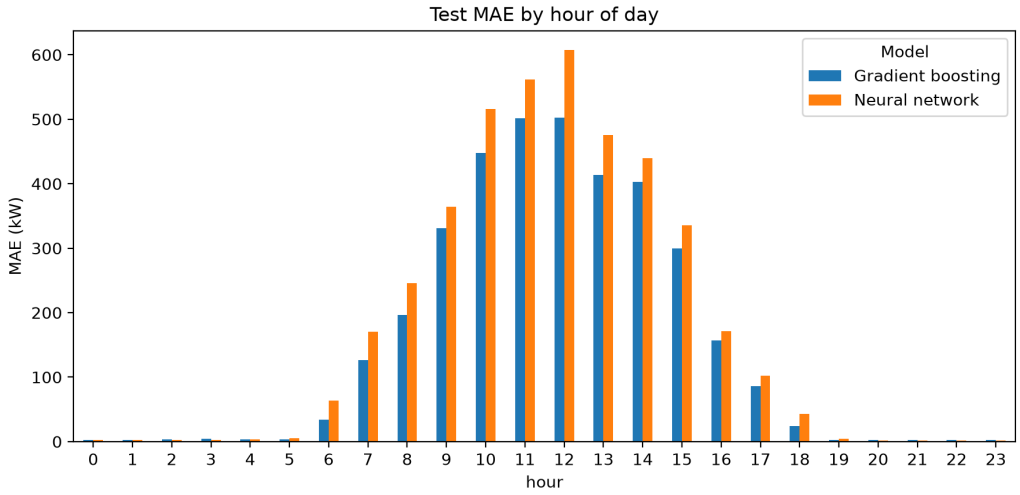


Figure 10: Test error by hour of the day for both models.

13.2 Error by sunlight level

Table 11 groups the test rows by the sunlight measured at the plant. The MAE grows with the sunlight level, but as a share of the mean output it stays at about 4 to 5% for gradient boosting and 5 to 7% for the neural network in all daytime bands. The models therefore do not get relatively worse in weak or strong sun; the absolute error grows because the output grows. The weakest daytime band has the highest relative error for the neural network (7.1%).

Sunlight (kW/m²)	Rows	Mean output	GB MAE	GB %	NN MAE	NN %
0 (night)	4,541	0	2.96	–	2.87	–
0–0.2	1,739	1,055	50.24	4.8	75.01	7.1
0.2–0.5	1,750	5,023	245.43	4.9	285.65	5.7
0.5–0.8	1,565	8,640	458.53	5.3	509.08	5.9
> 0.8	712	11,762	470.68	4.0	557.89	4.7

Table 11: Test MAE by sunlight level for gradient boosting (GB) and the neural network (NN). The % columns give the MAE as a share of the mean output in that band.

13.3 Largest errors

The largest errors are single events rather than a general weakness. The biggest one for both models is on 7 June 2020 at 12:45 for inverter 5 (Inv05): the sunlight was high (1.0 kW/m^2) and both models predicted about 12,600 to 12,700, but the inverter produced only 3,770. Another large overprediction is Inv19 at 13:45 on the same day (actual 572, gradient boosting 7,043). In

such cases the inputs say “strong sun” but the inverter output was low, so a model that only sees weather and time cannot predict the drop. Possible causes are an inverter fault, shading or a power limit, but the data does not say which.

The largest underpredictions go the other way. On 1 June 2020 at 11:15, several inverters (Inv03, Inv09 and Inv02) produced much more than predicted while the sensor showed only 0.49 kW/m^2, which suggests that the sensor reading at that moment did not match the sunlight reaching the panels. Several of the worst errors fall on the same timestamps and affect more than one inverter, which points to events in the plant or the sensor rather than to a weakness of one model.

14 Bias Check

The guide asks whether the model struggles with some groups. In the Kaggle data, the groups we can examine are the 22 inverters and the two months in the test set. We cannot check cities, because the Kaggle plant has no city column (see the Limitations section).

14.1 By inverter

Figure 11 shows the MAE of each inverter. For gradient boosting it ranges from 98 (Inv10) to 210 (Inv07), a factor of about two. As a share of each inverter’s mean output the range is smaller, 3.6% to 6.5%. The hardest inverters are the same for both models (Inv07 and Inv03 have the highest MAE for both, and Inv10 has the lowest for both), which suggests that the difference comes from the inverters’ data and not from one model. The inverters also differ in how much they produce: the mean test output ranges from 2,692 (Inv12) to 3,595 (Inv03), even though all of them share the same weather. The model only knows an inverter through its ID, so differences between panel groups can only be learned through that ID.

The signed bias per inverter is small. For gradient boosting the largest is +56 for Inv19, which is 1.7% of that inverter’s mean output; for the neural network it is −95 for Inv21 (3.2%). Inv19 is overpredicted by both models (+56 and +59) and appears several times in the list of largest errors, so it is the one inverter that deserves a closer look in the plant data.

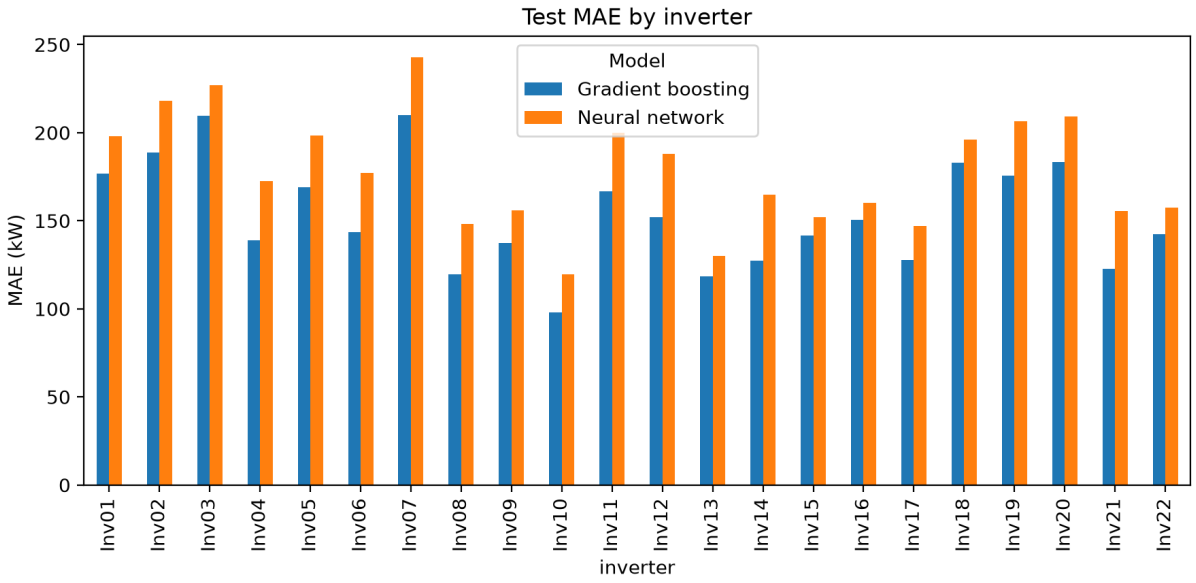


Figure 11: Test MAE for each of the 22 inverters.

14.2 By month

Only May and June 2020 appear in the data, so the month comparison has two groups (Figure 12). For gradient boosting the MAE is 170.59 in May and 137.89 in June; for the neural network it is 196.84 and 161.10. May has the higher mean output (3,358 against 2,968), and as a share of the mean output the errors are similar: 5.1% and 4.6% for gradient boosting, 5.9% and 5.4% for the neural network. The bias is close to zero for gradient boosting (+0.3 and +6.3) and slightly negative for the neural network (−32 and −22).

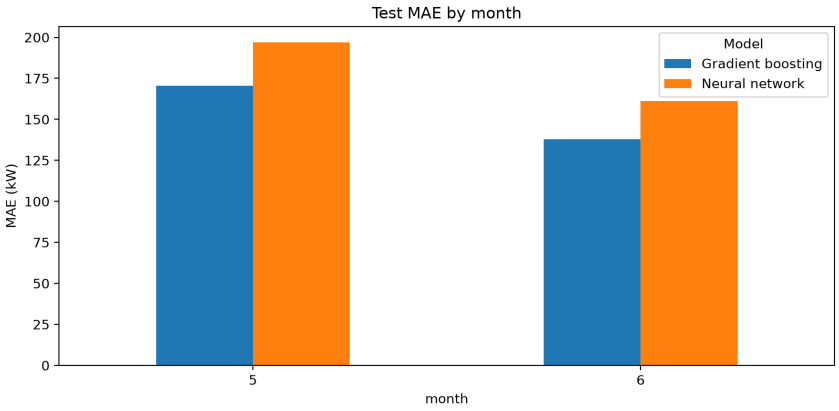


Figure 12: Test MAE by month for both models.

14.3 Summary of the bias check

We found no serious bias against any group we could test. No inverter is badly served: the signed bias per inverter is at most 1.7% (gradient boosting) and 3.2% (neural network) of its mean output. The two months have similar relative errors. The one systematic pattern is that the neural network underpredicts by about 1% on average, while gradient boosting does not. Two limits apply: the data covers only two months, so we cannot say how the models behave in winter, in the monsoon or under heavy smog, and we have no test for differences between cities.

15 Limitations

- **Random split and near-duplicate rows.** In the Kaggle data all 22 inverters are recorded at the same 15-minute timestamps, and every inverter row at a timestamp has the same weather sensor values. A random split therefore puts some rows of a timestamp in the training set and others in the test set, and neighbouring timestamps are also very similar. The model can partly "remember" the weather of a moment it has already seen, so our test scores are probably better than what we would get when forecasting a new day. A split by date (train on earlier days, test on later days) would be a stricter test and is left as future work.
- **One plant, one short period.** The Kaggle data comes from a single plant in India and covers only May and June 2020. We have no winter, monsoon or smog periods, and no Pakistani plant, so we could not test the model by city.
- **Inputs differ between training and the app.** The model is trained on sunlight, air temperature and module temperature measured at the plant. In the app the weather inputs will come from forecasts (NASA POWER and Open-Meteo), which have their own errors that are not included in our test results. Weather services also do not give module temperature, so the app will have to estimate it.

- **No direct cloud or smog input.** The Kaggle data has no cloud cover or PM2.5, so the model sees these effects only through the measured sunlight.
- **Scaling before the split.** The scaler for temperature and sunlight was fitted on all rows before the split. The effect is probably small, but a strict pipeline would fit it on the training set only.
- **Faults were removed.** Cleaning removed 63 rows with an inverter fault, so the model has never seen faulty behaviour and cannot predict sudden drops like the largest errors in the error analysis.
- **Limited tuning.** The neural network was tuned over only eight settings with 30 epochs each, so a better network may exist. The comparison with gradient boosting should be read with this in mind.
- **Easy binary task.** The high versus low task is easy (AUC-ROC about 1.00), so it does not separate the two models well.

16 Contribution Table

Table 12 links each Git commit author to the modules they wrote. All code was committed from each member's own account using the Git command line.

Git author	Member	Modules
AnooshaVohra, anushavohra (GitHub: anushavohra)	Member 1 (Anoosha Vohra)	Repository setup, `requirements.txt`, NASA POWER fetch and cleaning scripts, README, Deliverable 1 report source, `split_data.py` (70/15/15 split), `physics_baseline.py` (physics baseline)
kainat219	Member 2 (Kainat Afzal)	Kaggle data loader, generation cleaning, feature engineering, gradient boosting model (`model.py`), hyperparameter tuning (`tune.py`), PyTorch neural network (`nn_model.py`)
Manamha Nadeem (GitHub: Manumha-Nadeem)	Member 3 (Manumha Nadeem)	Open-Meteo fetch and merge, EDA script and charts, retailer list, `evaluate.py` (test metrics), `bias_check.py` (error by inverter, hour, month and irradiation), evaluation report sections and final LaTeX compilation

Table 12: Git commit author and the modules each member contributed.

Note: the Git author names come from each member's local Git configuration. "Manamha Nadeem" is a spelling error in Member 3's (Manumha Nadeem's) local Git configuration, and "AnooshaVohra" and "anushavohra" are two author names used by Member 1 with the same email address.

17 Next Steps

With the data pipeline built and the models trained and tested, the remaining work has four parts. First, we will test the models more strictly, by training on earlier days and testing on later days instead of splitting rows at random, and by giving them forecast weather instead of plant sensor readings. Second, we will estimate module temperature from air temperature and sunlight, so that the app can supply every input the model needs. Third, we will build the web app where a user selects a city and system size and sees the forecast output, expected savings

and nearby retailers from `data/retailers.csv`. Finally, we will add a chatbot that answers common questions about rooftop solar in Pakistan.